Task 16:

Configure an API Gateway to route client requests to appropriate microservices. Implement basic load balancing so that requests are distributed across multiple instances of a service.

Aim

To configure an **API Gateway** that routes client requests to appropriate microservices and implement **basic load balancing** to distribute requests across multiple instances of a service.

Algorithm

1. Create a new Spring Boot project → **API Gateway**.
2. Add dependencies:
    - Spring Cloud Gateway
    - Eureka Client
3. Enable service discovery in Gateway.
4. Configure routes using application properties.
5. Use **logical service names** instead of URLs.
6. Enable **load balancing** using lb://SERVICE-NAME.
7. Run multiple instances of a service (e.g., Product Service on different ports).
8. Start:
    - Eureka Server
    - Product Service (multiple instances)
    - Order Service
    - API Gateway
9. Send requests through Gateway.
10. Verify routing and load balancing.

Architecture

Client → API Gateway → Microservices (via Eureka)

Codes:

API Gateway Project

Dependencies (pom.xml)

```
<dependency>
   <groupId>org.springframework.cloud</groupId>
   <artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>

<dependency>
   <groupId>org.springframework.cloud</groupId>
   <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

Main Class

```
package com.gateway;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ApiGatewayApplication {

   public static void main(String[] args) {
      SpringApplication.run(ApiGatewayApplication.class, args);
   }
}
```

application.properties

```
server.port=8080

spring.application.name=API-GATEWAY

eureka.client.service-url.defaultZone=http://localhost:8761/eureka

spring.cloud.gateway.routes[0].id=product-service
spring.cloud.gateway.routes[0].uri=lb://PRODUCT-SERVICE
spring.cloud.gateway.routes[0].predicates[0]=Path=/products/**
```

```
spring.cloud.gateway.routes[1].id=order-service
spring.cloud.gateway.routes[1].uri=lb://ORDER-SERVICE
spring.cloud.gateway.routes[1].predicates[0]=Path=/orders/**
```

Output:


```
[
  {
    "id": 1,
    "name": "Laptop",
    "price": 50000
  }
]
```


Result

API Gateway successfully routes requests Hardcoded URLs removed (uses Eureka) Load balancing achieved (Round Robin) Scalable and flexible architecture